

Title Page

Course: CO2 - Algorithm Analysis and Design

Assessment: AT3

Question No: 7

Title: Brute- Force Approach to Find the Farthest Pair of Delivery Points

Name: ____ARCHANA DEVI.P_____

Register Number: ____192521192_____

Department: Information Technology

Date: ____30.07.2026_____

Aim

To identify and correct the errors in the given Insertion Sort algorithm, explain the debugging process, analyze its time complexity, and verify the corrected output.

Problem Statement

The given Insertion Sort program contains logical errors in the loop condition and insertion statement, causing incorrect sorting. The objective is to identify these errors, correct them, explain the debugging process, and analyze the algorithm.

Given Incorrect Code

Type the following exactly as provided in the question.

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
        while j > 0 and arr[j] > key: # ERROR  
            arr[j + 1] = arr[j]  
            j = j - 1  
        arr[j] = key # ERROR  
    return arr
```

Errors Identified

Error 1: Incorrect Loop Condition

Incorrect Code

```
while j > 0 and arr[j] > key:
```

Explanation

The condition $j > 0$ prevents the algorithm from comparing the element at index 0. As a result, the first element may never be moved, leading to an incorrectly sorted array.

Correct Code

```
while j >= 0 and arr[j] > key:
```

Error 2: Incorrect Insertion Position

Incorrect Code

```
arr[j] = key
```

Explanation

After the shifting process, j is decreased one extra time. Therefore, the correct position for the key is $j + 1$.

Correct Code

```
arr[j + 1] = key
```

Step- by- Step Debugging

Initial Array

```
[12, 11, 13, 5, 6]
```

Pass 1

Key = 11

12 is greater than 11, so shift 12.

```
[12, 12, 13, 5, 6]
```

Insert 11.

```
[11, 12, 13, 5, 6]
```

Pass 2

Key = 13

13 is already greater than 12.

No shifting required.

```
[11, 12, 13, 5, 6]
```

Pass 3

Key = 5

Shift 13.

Shift 12.

Shift 11.

Insert 5.

[5, 11, 12, 13, 6]

Pass 4

Key = 6

Shift 13.

Shift 12.

Shift 11.

Insert 6.

[5, 6, 11, 12, 13]

Corrected Algorithm

Start

Repeat from second element to last element

Store current element as Key

Compare Key with previous elements

Shift all larger elements one position to the right

Insert Key at its correct position

Repeat until the array is sorted

Stop

Corrected Python Program

<pre> main.c 1 #include <stdio.h> 2 3 void insertionSort(int arr[], int n) { 4 int i, key, j; 5 6 for (i = 1; i < n; i++) { 7 key = arr[i]; 8 j = i - 1; 9 10 while (j >= 0 && arr[j] > key) { 11 arr[j + 1] = arr[j]; 12 j--; 13 } 14 15 arr[j + 1] = key; 16 } 17 } 18 19 int main() { 20 int arr[] = {12, 11, 13, 5, 6}; 21 int n = sizeof(arr) / sizeof(arr[0]); 22 23 printf("Original Array: "); 24 for (int i = 0; i < n; i++) 25 printf("%d ", arr[i]); 26 </pre>	<pre> main.c 9 10 while (j >= 0 && arr[j] > key) { 11 arr[j + 1] = arr[j]; 12 j--; 13 } 14 15 arr[j + 1] = key; 16 } 17 } 18 19 int main() { 20 int arr[] = {12, 11, 13, 5, 6}; 21 int n = sizeof(arr) / sizeof(arr[0]); 22 23 printf("Original Array: "); 24 for (int i = 0; i < n; i++) 25 printf("%d ", arr[i]); 26 27 insertionSort(arr, n); 28 29 printf("\nSorted Array: "); 30 for (int i = 0; i < n; i++) 31 printf("%d ", arr[i]); 32 33 return 0; 34 } </pre>
---	---

Output

Original Array : [12, 11, 13, 5, 6]

Sorted Array : [5, 6, 11, 12, 13]

Time Complexity

Case	Complexity
------	------------

Best Case	$O(n)$
-----------	--------

Average Case	$O(n^2)$
--------------	----------

Worst
Case $O(n^2)$

Space Complexity
 $O(1)$

Readability Improvements

- Proper indentation is maintained.
- Comments are added for better understanding.
- Variable names are clear and meaningful.
- The program is divided into logical sections.
- The code is easier to read and maintain.

Conclusion

The errors in the original Insertion Sort program were successfully identified and corrected. After fixing the loop condition and insertion position, the algorithm correctly sorts the array. The program produces the expected output with a time complexity of $O(n^2)$ in the average and worst cases and $O(n)$ in the best case.